```
def is_variable(x):
    """Returns True if x is a variable (represented as a lowercase string)."""
    return isinstance(x, str) and x.islower()

def is_constant(x):
    """Returns True if x is a constant (represented as a capitalized/uppercase string)."""
    return isinstance(x, str) and not x.islower()

def occurs_check(var, term, subst):
    """Returns True if variable 'var' occurs anywhere inside 'term'."""
    if var == term:
        return True
    if is_variable(term) and term in subst:
        return occurs_check(var, subst[term], subst)
    if isinstance(term, (list, tuple)):
        return any(occurs_check(var, arg, subst) for arg in term)
    return False

def unify(x, y, subst=None):
    """
    Finds the Most General Unifier (MGU) for two FOL expressions x and y.
    Returns a dictionary of substitutions, or None if unification fails.
    """
    if subst is None:
        subst = {}

    if subst is None:
        return None

    if x == y:
        return subst

    if is_variable(x):
        return unify_variable(x, y, subst)

    if is_variable(y):
        return unify_variable(y, x, subst)

    if isinstance(x, (list, tuple)) and isinstance(y, (list, tuple)):
        if len(x) != len(y):
            return None

        for arg_x, arg_y in zip(x, y):
            subst = unify(arg_x, arg_y, subst)
            if subst is None:
                return None
        return subst

    return None

def unify_variable(var, x, subst):
    """Helper function to handle variable bindings."""
    if var in subst:
        return unify(subst[var], x, subst)

    if is_variable(x) and x in subst:
        return unify(var, subst[x], subst)

    if occurs_check(var, x, subst):
        return None

    new_subst = subst.copy()
    new_subst[var] = x
    return new_subst

def check_unification(expr1, expr2):
    """
    Wrapper function that explicitly states if two expressions
    can unify successfully and prints the substitutions.
    """
    print(f"Expression 1: {expr1}")
    print(f"Expression 2: {expr2}")

    result = unify(expr1, expr2)

    if result is not None:
        print("Result      : Unified ")
        print(f"Substitution: {result}")
    else:
        print("Result      : not unify")
    print("-" * 50)
```

```
    print(      )


# ========================================
# Running the Tests
# ========================================
if __name__ == "__main__":
    # Test Case 1: Standard Success
    e1 = ("Knows", "John", "x")
    e2 = ("Knows", "John", "Jane")
    check_unification(e1, e2)

    # Test Case 2: Constant Mismatch Failure
    e3 = ("Loves", "John", "x")
    e4 = ("Hates", "John", "Bob")
    check_unification(e3, e4)

    # Fixed: Added a comma after "x"
    e5 = ("P", "b", "x", ("f", ("g", "z")))  # Represents P(b, x, f(g(y)))
    e6 = ("P", "z", ("f", "y"), ("f", "y"))  # Represents P(z, f(y), f(y))
    check_unification(e5, e6)

    # Test Case 4: Occurs Check Failure
    e7 = ("P", "x")
    e8 = ("P", ("f", "x"))
    check_unification(e7, e8)
```

```
Expression 1: ('Knows', 'John', 'x')
Expression 2: ('Knows', 'John', 'Jane')
Result      : Unified
Substitution: {'x': 'Jane'}
--------------------------------------------------
Expression 1: ('Loves', 'John', 'x')
Expression 2: ('Hates', 'John', 'Bob')
Result      : not unify
--------------------------------------------------
Expression 1: ('P', 'b', 'x', ('f', ('g', 'z')))
Expression 2: ('P', 'z', ('f', 'y'), ('f', 'y'))
Result      : Unified
Substitution: {'b': 'z', 'x': ('f', 'y'), 'y': ('g', 'z')}
--------------------------------------------------
Expression 1: ('P', 'x')
Expression 2: ('P', ('f', 'x'))
Result      : not unify
--------------------------------------------------
```